Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа № 3

За седьмой семестр

Тема: «Многослойный персептрон для XOR с ReLU на скрытых слоях: сравнение активационных функций»

Выполнил:

Студент 4 курса

Группы ИИ-26

Хомиченко И.А.

Проверил:

Никитин С.К.

2026

Цель работы: На основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовать влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провести сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.

-

Вариант 7 (c0 = −4, c1 = −9)

A	B	A ⊕ B
−4	−4	−4
−4	−9	−9

−9	−4	−9
−9	−9	−4

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 с функцией потерь BCE из ЛР №2.
2. Реализовать активационную функцию ReLU на скрытом слое и осуществить корректное обратное распространение ошибки для ReLU. На выходном слое оставить Сигмоиду.
3. Обучить сеть на полной таблице истинности в двух конфигурациях:
   - **Конфигурация A (базовая):** MLP 2-2-1, Сигмоида на всех слоях, BCE, нормализованные целевые значения (0/1);
   - **Конфигурация B (новая):** MLP 2-2-1, ReLU на скрытом слое, Сигмоида на выходном, BCE, нормализованные (0/1).
4. Критерий остановки для обеих конфигураций: суммарная ошибка Es ≤ Ee (как в ЛР №2).
5. Для каждой конфигурации зафиксировать:
   - Количество эпох до критерия остановки.
   - Итоговую суммарную ошибку и accuracy.
   - Среднюю абсолютную ошибку в исходной шкале [c0; c1].
6. Провести не менее 5 запусков обучения для каждой конфигурации с разными seed и сравнить устойчивость.
7. Построить графики:
   - График сходимости (суммарная ошибка Es от эпохи) для обеих конфигураций на одних осях.
   - Диаграмма разброса числа эпох по 5 запускам (столбики) для конфигурации A и B.
8. Визуализация разделяющей поверхности:
   - Для представительного экземпляра каждой конфигурации построить карту (heatmap) выходного значения сети на плоскости (A, B) ∈ [−10; 10]² с наложением 4 точек обучающей выборки.
   - Описать, как ReLU изменяет форму разделяющей поверхности (кусочно-линейные границы vs гладкие).

9. Реализовать режим функционирования сети:
   - Пользователь вводит (A, B) из [−10; 10].
   - Сеть выводит: выход в нормализованной шкале, выход в исходной шкале [c0; c1], класс.
   - Продемонстрировать на 4 обучающих примерах + 2–3 дополнительных парах.
10. Создать таблицу сравнения (статистика по 5 запускам, MAE для обеих конфигураций).
11. Вывод: ответить на все вопросы и провести анализ результатов

Код программы:

```
from pathlib import Path
import csv
import numpy as np
import matplotlib.pyplot as plt

C0, C1 = -4.0, -9.0
SEEDS = [0, 1, 2, 3, 4]
LR = 0.7
MAX_EPOCHS = 20000
EE = 0.20
OUT_DIR = Path(__file__).resolve().parent / 'results'
OUT_DIR.mkdir(exist_ok=True)

X_RAW = np.array([[-4,-4],[-4,-9],[-9,-4],[-9,-9]], dtype=float)
Y = np.array([[0.],[1.],[1.],[0.]], dtype=float)
X = (X_RAW - C0) / (C1 - C0)

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -60, 60)))

def relu(z):
    return np.maximum(z, 0)

def init_weights(seed):
    rng = np.random.RandomState(seed)
    return [rng.uniform(-0.5,0.5,(2,2)), np.zeros((1,2)),
            rng.uniform(-0.5,0.5,(2,1)), np.zeros((1,1))]

def forward(x, p, mode):
    w1,b1,w2,b2 = p
    z1 = x @ w1 + b1
    h = sigmoid(z1) if mode == 'A' else relu(z1)
    yhat = sigmoid(h @ w2 + b2)
    return z1,h,yhat

def loss(yhat):
    p = np.clip(yhat, 1e-12, 1-1e-12)
    return float(-np.sum(Y*np.log(p)+(1-Y)*np.log(1-p)))

def train(mode, seed):
    p = init_weights(seed)
    hist = []
    stop_epoch = None
    for epoch in range(MAX_EPOCHS+1):
        current_loss = loss(forward(X,p,mode)[2])
        hist.append(current_loss)
        if current_loss <= EE:
            stop_epoch = epoch
```

```
            break
        if epoch == MAX_EPOCHS:
            break

        for i in range(len(X)):
            x = X[i:i+1]
            t = Y[i:i+1]
            z,h,yhat = forward(x,p,mode)
            w1,b1,w2,b2 = p
            delta_out = yhat - t  # Sigmoid + BCE
            deriv = h*(1-h) if mode == 'A' else (z > 0).astype(float)
            delta_hidden = (delta_out @ w2.T) * deriv
            w2 -= LR * (h.T @ delta_out)
            b2 -= LR * delta_out
            w1 -= LR * (x.T @ delta_hidden)
            b1 -= LR * delta_hidden
    predicted = forward(X,p,mode)[2].reshape(-1)
    labels = np.where(predicted >= .5, C1, C0)
    restored = C0 + predicted*(C1-C0)
    mae = float(np.mean(np.abs(restored-(C0+Y.reshape(-1)*(C1-C0)))))
    return dict(mode=mode,seed=seed,params=p,history=hist,epochs=len(hist)-1,
                converged=stop_epoch is not None,es=hist[-1],
                accuracy=float(np.mean(labels==(C0+Y.reshape(-1)*(C1-
C0)))),mae=mae)

def predict(run, a, b):
    if not (-10 <= a <= 10 and -10 <= b <= 10):
        raise ValueError('A и B должны находиться в диапазоне [-10; 10].')
    x = (np.array([[a,b]],float)-C0)/(C1-C0)
    v = float(forward(x,run['params'],run['mode'])[2][0,0])
    return v, C0+v*(C1-C0), int(C1 if v >= 0.5 else C0)

def pick(runs, mode):
    rr = [r for r in runs if r['mode']==mode]
    good = [r for r in rr if r['converged'] and r['accuracy'] == 1]
    return min(good or rr, key=lambda r:(r['mae'],r['es']))

def plots(runs):
    a,b = pick(runs,'A'),pick(runs,'B')
    fig,ax=plt.subplots(figsize=(9,5))
    ax.plot(a['history'],label=f"A Sigmoid, seed={a['seed']}")
    ax.plot(b['history'],label=f"B ReLU, seed={b['seed']}")
    ax.axhline(EE,c='gray',ls='--',label=f'Ee={EE}')
    ax.set(xlabel='Epoch',ylabel='Es = sum BCE',title='Convergence of two MLP
configurations')
    ax.set_yscale('log');ax.legend();fig.tight_layout()
    fig.savefig(OUT_DIR/'01_convergence.png',dpi=180);plt.close(fig)

    fig,ax=plt.subplots(figsize=(9,5))
    q=np.arange(len(SEEDS)); w=.36
    for mode,shift,color in [('A',-w/2,'tab:blue'),('B',w/2,'tab:orange')]:
        s=[next(r for r in runs if r['mode']==mode and r['seed']==seed) for
seed in SEEDS]
        bars=ax.bar(q+shift,[r['epochs'] for r in
s],w,label=mode,color=color)
        for bar,r in zip(bars,s):
            if not r['converged']:

ax.text(bar.get_x()+bar.get_width()/2,bar.get_height(),'*',ha='center',va='bo
ttom',fontsize=14)
    ax.set(xticks=q,xticklabels=[str(s) for s in
SEEDS],xlabel='Seed',ylabel='Epochs',title='Epoch counts over 5 random
seeds')
    ax.legend(title='Configuration');ax.text(.01,-.17,'* = maximum epochs,
```

```
stopping criterion not reached',transform=ax.transAxes,fontsize=9)

fig.tight_layout();fig.savefig(OUT_DIR/'02_epochs.png',dpi=180,bbox_inches='t
ight');plt.close(fig)

    grid=np.linspace(-10,10,201);ga,gb=np.meshgrid(grid,grid)
    x=(np.column_stack([ga.ravel(),gb.ravel()])-C0)/(C1-C0)
    fig,axes=plt.subplots(1,2,figsize=(12,5),sharex=True,sharey=True)
    for ax,r,title in zip(axes,[a,b],['A: Sigmoid hidden','B: ReLU hidden']):
        values=forward(x,r['params'],r['mode'])[2].reshape(ga.shape)
        heat=ax.imshow(values,origin='lower',extent=[-10,10,-
10,10],vmin=0,vmax=1,cmap='viridis',aspect='equal')
        if values.min()<0.5<values.max():

ax.contour(ga,gb,values,levels=[.5],colors=['white'],linewidths=1.5)

ax.scatter(X_RAW[:,0],X_RAW[:,1],c=Y.ravel(),cmap='coolwarm',vmin=0,vmax=1,s=
80,edgecolors='black',linewidths=1)
        ax.set(xlabel='A',ylabel='B',title=f"{title} (seed={r['seed']})")
    fig.colorbar(heat,ax=axes.ravel().tolist(),label='Normalized
output',shrink=.82)
    fig.subplots_adjust(left=.07,right=.88,bottom=.12,top=.88,wspace=.15)
    fig.savefig(OUT_DIR/'03_heatmaps.png',dpi=180);plt.close(fig)

    fig,ax=plt.subplots(figsize=(7,4.5))
    means=[np.mean([r['mae'] for r in runs if r['mode']==m]) for m in 'AB']
    bars=ax.bar(['A: Sigmoid','B: ReLU'],means)
    for bar,v in zip(bars,means):

ax.text(bar.get_x()+bar.get_width()/2,bar.get_height(),f'{v:.4f}',ha='center'
,va='bottom')
    ax.set(ylabel='Mean MAE (original [-9;-4] scale)',title='MAE averaged
over 5 runs')

fig.tight_layout();fig.savefig(OUT_DIR/'04_mae.png',dpi=180);plt.close(fig)

def print_summary(runs):
    print(f'Variant 7: C0={C0:g}, C1={C1:g}; criterion Es<=Ee={EE}; max
epochs={MAX_EPOCHS}')
    print('Mode seed epochs reached Es accuracy MAE')
    for r in runs:
        print(f"{r['mode']:>4} {r['seed']:>4} {r['epochs']:>6}
{str(r['converged']):>7} {r['es']:>9.6f} {r['accuracy']:>8.2%}
{r['mae']:>8.5f}")
    for mode in 'AB':
        subset=[r for r in runs if r['mode']==mode]
        print(f"{mode}: converged {sum(r['converged'] for r in subset)}/5;
mean epochs={np.mean([r['epochs'] for r in subset]):.1f}, mean
MAE={np.mean([r['mae'] for r in subset]):.4f}, mean
accuracy={np.mean([r['accuracy'] for r in subset]):.1%}")
    print('\nPredictions on 4 training + 3 additional pairs (best runs):')
    for mode in 'AB':
        r=pick(runs,mode)
        print(f"CONFIG {mode}, seed {r['seed']}")
        for aa,bb in [(-4,-4),(-4,-9),(-9,-4),(-9,-9),(-6,-6),(-10,10),(0,-
5)]:
            o,raw,cls=predict(r,aa,bb)
            print(f'A={aa:5g} B={bb:5g} => norm={o:.5f}, original={raw:.5f},
class={cls}')

def write_csv(runs):
    with (OUT_DIR/'statistics.csv').open('w',encoding='utf-8-sig',newline='')
as f:
        wr=csv.writer(f,delimiter=';')
```

```
wr.writerow(['configuration','seed','epochs','reached_Ee','Es','accuracy','MA
E'])
        for r in runs:

wr.writerow([r['mode'],r['seed'],r['epochs'],r['converged'],f"{r['es']:.8f}",
f"{r['accuracy']:.6f}",f"{r['mae']:.8f}"])

def interactive(runs):
    print('\nInteractive mode: enter A B, or q to exit.')
    while True:
        raw=input('A B > ').strip()
        if raw.lower() in ('q','quit','exit'): break
        try:
            numbers=raw.replace(';',' ').replace(',',' ').split()
            if len(numbers)!=2: raise ValueError('Enter exactly two
numbers.')
            aa,bb=map(float,numbers)
            for mode in 'AB':
                norm,orig,cls=predict(pick(runs,mode),aa,bb)
                print(f'{mode}: normalized={norm:.6f}; original={orig:.6f};
class={cls}')
        except ValueError as e:
            print('Input error:',e)

if __name__=='__main__':
    all_runs=[train(mode,seed) for mode in 'AB' for seed in SEEDS]
    print_summary(all_runs)
    write_csv(all_runs)
    plots(all_runs)
    print(f'Graphs and CSV saved to: {OUT_DIR}')
    interactive(all_runs)
```

Результаты программы:

Variant 7: C0=-4, C1=-9; criterion Es<=Ee=0.2; max epochs=20000

Mode seed epochs reached Es accuracy MAE

A 0 1862 True 0.199813 100.00% 0.24202

A 1 338 True 0.199373 100.00% 0.24253

A 2 429 True 0.198149 100.00% 0.24096

A 3 690 True 0.199483 100.00% 0.24226

A 4 387 True 0.199447 100.00% 0.24261

B 0 20000 False 1.909702 75.00% 1.66066

B 1 20000 False 2.773236 50.00% 2.50000

B 2 20000 False 2.773236 50.00% 2.50000

B 3 20000 False 1.909702 75.00% 1.66066

B 4 20000 False 2.773236 50.00% 2.50000

A: converged 5/5; mean epochs=741.2, mean MAE=0.2421, mean accuracy=100.0%

B: converged 0/5; mean epochs=20000.0, mean MAE=2.1643, mean accuracy=60.0%

Predictions on 4 training + 3 additional pairs (best runs):

CONFIG A, seed 2

A= -4 B= -4 => norm=0.02911, original=-4.14555, class=-4

A= -4 B= -9 => norm=0.95692, original=-8.78461, class=-9

A= -9 B= -4 => norm=0.95321, original=-8.76603, class=-9

A= -9 B= -9 => norm=0.07378, original=-4.36892, class=-4

A= -6 B= -6 => norm=0.96694, original=-8.83472, class=-9

A= -10 B= 10 => norm=0.01537, original=-4.07687, class=-4

A=   0 B= -5 => norm=0.01553, original=-4.07765, class=-4

CONFIG B, seed 3

A= -4 B= -4 => norm=0.32852, original=-5.64262, class=-4

A= -4 B= -9 => norm=0.32852, original=-5.64262, class=-4

A= -9 B= -4 => norm=1.00000, original=-8.99999, class=-9

A= -9 B= -9 => norm=0.32852, original=-5.64262, class=-4

A= -6 B= -6 => norm=0.32852, original=-5.64262, class=-4

A= -10 B= 10 => norm=1.00000, original=-9.00000, class=-9

A=   0 B= -5 => norm=0.32852, original=-5.64262, class=-4

Graphs and CSV saved to: C:\Users\fhdgsjfbdjsg\Desktop\MRZIS3\results

Interactive mode: enter A B, or q to exit.

A B > -4 4

A: normalized=0.015374; original=-4.076870; class=-4

B: normalized=1.000000; original=-9.000000; class=-9

A B > -4 -9

A: normalized=0.956921; original=-8.784605; class=-9

B: normalized=0.328523; original=-5.642617; class=-4

A B > -9 -4

A: normalized=0.953206; original=-8.766032; class=-9

B: normalized=0.999998; original=-8.999988; class=-9

A B > -9 -9

A: normalized=0.073784; original=-4.368920; class=-4

B: normalized=0.328523; original=-5.642617; class=-4

A B > -5 -8

A: normalized=0.956222; original=-8.781111; class=-9

B: normalized=0.328523; original=-5.642617; class=-4

A B > -7 -7

A: normalized=0.893213; original=-8.466065; class=-9

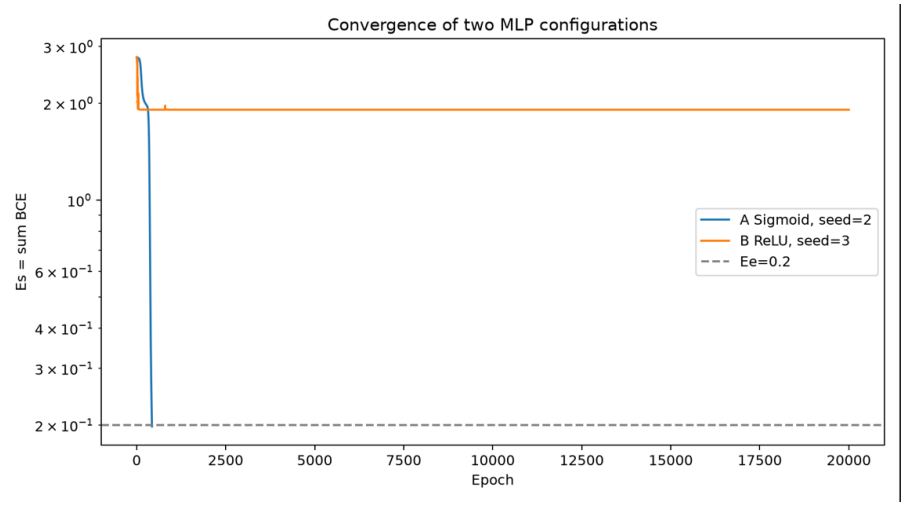
B: normalized=0.328523; original=-5.642617; class=-4

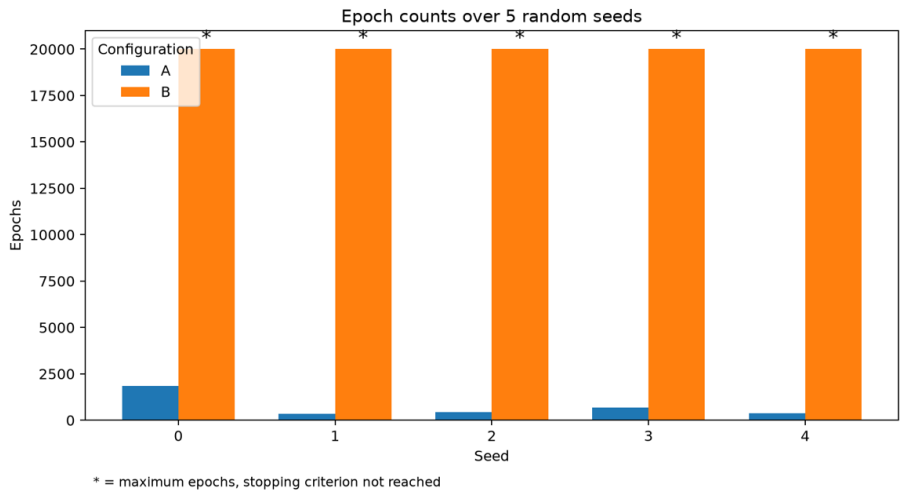
A B > -10 -10

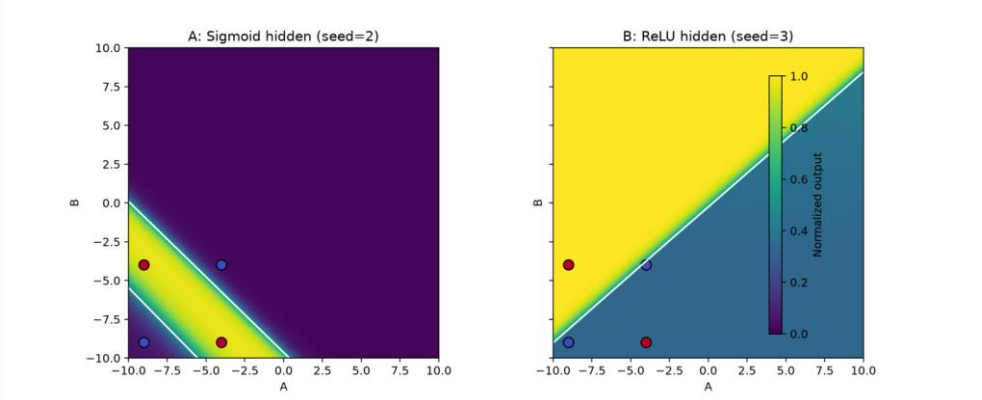
A: normalized=0.043858; original=-4.219291; class=-4

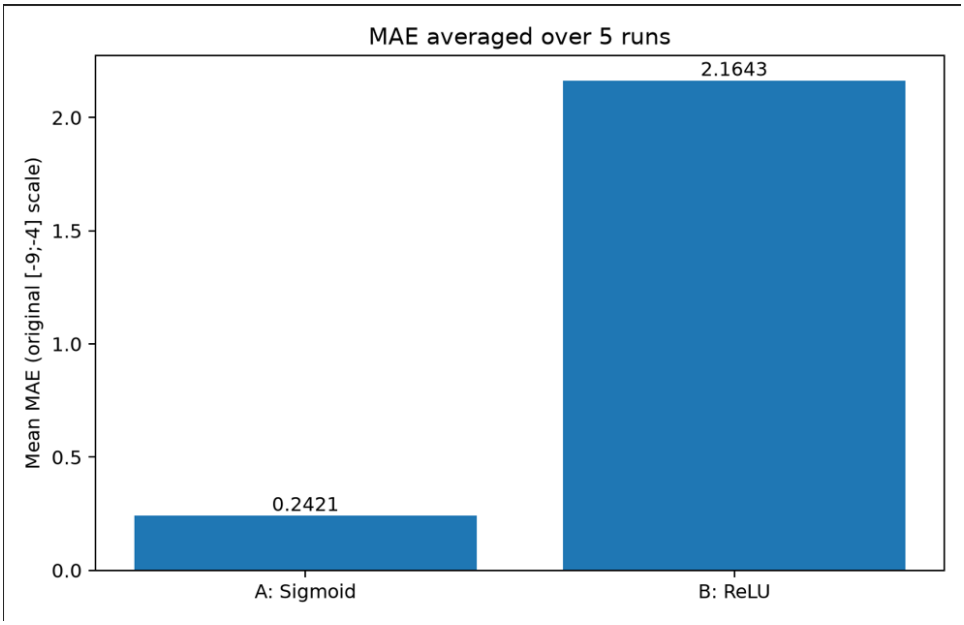
B: normalized=0.328523; original=-5.642617; class=-4

A B > q









В ходе лабораторной работы была реализована и обучена нейронная сеть MLP 2-2-1 для решения задачи XOR при c0 = -4 и c1 = -9. Проведено сравнение двух конфигураций: с сигмоидальной функцией активации и с ReLU на скрытом слое. Обучение выполнялось с использованием функции потерь BCE и нормализованных значений 0 и 1. Конфигурация с Сигмоидой показала более устойчивые результаты, достигнув критерия остановки во всех пяти запусках и точности 100%. Конфигурация с ReLU оказалась более чувствительной к начальным весам и не достигла заданного порога ошибки. Таким образом, для данной задачи сигмоидальная функция активации показала лучшую устойчивость обучения.

Вывод: На основе многослойного персептрона архитектуры 2-2-1 из ЛР №2, исследовал влияние активационной функции ReLU (Rectified Linear Unit) на скрытых слоях. Провел сравнительный анализ обучения сети с функцией активации Сигмоида (базовая конфигурация) и ReLU (новая конфигурация) по скорости сходимости, качеству обучения, форме разделяющей поверхности и устойчивости к инициализации весов.